Chapter 12: More on Classes

ENG3091

Khiem Nguyen

Contents

1 The hidden `this` pointer **2**
- 1.1 From `self` to `this` . . . 2
- 1.2 The hidden `this` pointer . . . 2
- 1.3 How is `this` set? . . . 3
- 1.4 When to write `this` explicitly . . . 3
- 1.5 Returning `*this`: method chaining . . . 3
- 1.6 Method chaining in action . . . 4
- 1.7 Method chaining, in a picture . . . 4

2 Destructors **5**
- 2.1 The cleanup problem . . . 5
- 2.2 AddressSanitizer agrees . . . 5
- 2.3 A `cleanup()` function? . . . 6
- 2.4 Destructors to the rescue . . . 6
- 2.5 A first destructor . . . 7
- 2.6 When do destructors run? . . . 7
- 2.7 The database helper, fixed . . . 8
- 2.8 RAII: let objects clean up after themselves . . . 8
- 2.9 A class template that owns an array . . . 9
- 2.10 Watch out: copying an object that owns memory . . . 10
- 2.11 What went wrong: a shallow copy . . . 10
- 2.12 The rule of three (and of zero) . . . 11

3 Class templates **11**
- 3.1 The same problem, for classes . . . 11
- 3.2 Class templates . . . 11
- 3.3 What the compiler generates . . . 13
- 3.4 Class templates and function templates together . . . 14
- 3.5 Several template types . . . 15
- 3.6 `std::pair`: the standard library's version . . . 15

4 Friends **16**
- 4.1 Why friends? . . . 16
- 4.2 Friendship . . . 16
- 4.3 A friend non-member function . . . 17
- 4.4 Defining a friend inside the class . . . 18
- 4.5 A friend of several classes . . . 19
- 4.6 Prefer non-friend functions . . . 20
- 4.7 …when a public member will do . . . 20
- 4.8 Friend classes . . . 21
- 4.9 Friendship has limits . . . 21

4.10 Friend member functions (self-study) . . . 22
4.11 Attempt 1: the friend's class isn't defined yet . . . 22
4.12 Attempt 2: swap the classes around . . . 23
4.13 Attempt 3: declare first, define later . . . 24

5 Wrapping up **24**
5.1 Summary . . . 24
5.2 Keep learning . . . 25

1 The hidden `this` pointer

1.1 From `self` to `this`

In Python, every method's first parameter, by convention called `self`, refers to the object the method was called on:

```python
class Fraction:
    def __init__(self, numerator, denominator):
        self.numerator = numerator
        self.denominator = denominator
```

In Chapter 11 we met C++'s version, `this`. Now let's look at it properly: it plays the role of `self`, but it's a **pointer**, and you never write it as a parameter.

1.2 The hidden `this` pointer

Listing 1 `hidden_pointer_this.cpp`

```cpp
#include <iostream>

class Simple {
  private:
    double value{};

  public:
    Simple(double value) : value{value} {}

    double get_value() const { return value; } // same as this->value
    void set_value(double value) { this->value = value; }
    void print() const { std::cout << this->value << '\n'; }
};

int main() {
    Simple simple{0.0};
    std::cout << simple.get_value() << '\n';
    simple.set_value(42.5);
    simple.print();
    return 0;
}
```

```
0
42.5
```

Inside every member function, **`this`** holds the address of the implicit object. Select its members with `->` (Chapter 11).

1.3 How is `this` set?

When we write `simple.set_value(42.5);`, the compiler (conceptually) turns it into an ordinary function call, with the object's address as an extra, hidden argument:

```
Simple::set_value(&simple, 42.5); // not real C++: just what happens underneath
```

So in each call, `this` points at whichever object the function was called on:

```
Simple a{1.0}; // inside the constructor: this == &a
Simple b{2.0}; // inside the constructor: this == &b
a.set_value(12); // inside set_value():  this == &a
b.set_value(21); // inside set_value():  this == &b
```

In a `const` member function, `this` points to a `const` object: that's where Chapter 10's error message "*passing 'const Date' as 'this' argument*" came from.

1.4 When to write `this` explicitly

Most of the time we don't: a plain `value` already means `this->value`. We need it when a parameter has the **same name** as a member:

```
struct Something {
    double data{};

    void set_data(double data) {
        this->data = data; // this->data is the member; data is the parameter
    }
};
```

1.5 Returning `*this`: method chaining

You've been chaining calls since Chapter 01:

```
std::cout << "Hello, " << user_name;
// is evaluated as
(std::cout << "Hello, ") << user_name;
```

`(std::cout << "Hello, ")` prints, and then **returns `std::cout` itself**, ready for the next `<<`. Our member functions can do the same: return `*this`, the implicit object, by reference.

This is called **method chaining** (or *function chaining*). We'll come back to it in Chapter 13, when we write our own `<<`.

1.6 Method chaining in action

Listing 2 `method_chaining_example.cpp`

```cpp
#include <iostream>

class Calculator {
  private:
    double value{};

  public:
    Calculator(double value) : value{value} {}

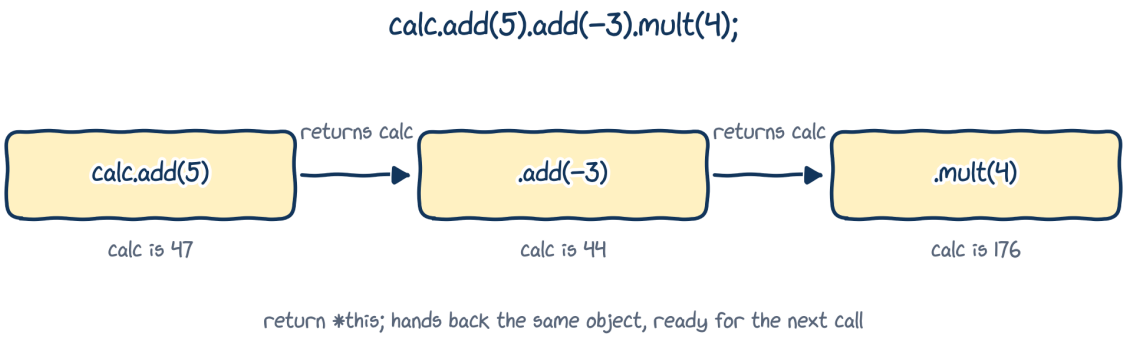
    Calculator& add(double x) {
        value += x;
        return *this; // return the implicit object itself
    }
    Calculator& mult(double x) {
        value *= x;
        return *this;
    }
    double get_value() const { return value; }
};

int main() {
    Calculator calc{42};
    calc.add(5).add(-3).mult(4); // method chaining
    std::cout << calc.get_value() << '\n';
    return 0;
}
```

```
176
```

Each function returns a **reference** to the same object. Return by value instead, and each call would work on a copy!

1.7 Method chaining, in a picture



`calc.add(5)` changes `calc` and returns `calc` itself, so `.add(-3)` is called on `calc` too, and so on. That's exactly how `std::cout << a << b` works.

2 Destructors

2.1 The cleanup problem

Remember memory leaks from Chapter 09? Here's a class that allocates memory when it's created:

Listing 3 `database_helper_without_destructor.cpp`

```
#include <iostream>
#include <string>

class DatabaseHelper {
  private:
    std::string* connection{}; // allocated on the heap

  public:
    DatabaseHelper(const std::string& path)
        : connection{new std::string{path}} {
        std::cout << "Connected to " << *connection << '\n';
    }
};

int main() {
    {
        DatabaseHelper db{"127.0.0.1:5432/my_database"};
    } // db is destroyed here... but nobody deletes *connection
    return 0;
}
```

2.2 AddressSanitizer agrees

```
$ g++ -std=c++20 -g -fsanitize=address database_helper_without_destructor.cpp
$ ./a.out
Connected to 127.0.0.1:5432/my_database
==3376==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 32 byte(s) in 1 object(s) allocated from:
    #1 in DatabaseHelper::DatabaseHelper(const std::string&) ...cpp:10
SUMMARY: AddressSanitizer: 59 byte(s) leaked in 2 allocation(s).
```

(Output shortened.) `db` is destroyed at the end of its block, but the `std::string` it created with `new` on line 10 is never deleted. And `connection` is private, so `main()` can't delete it either.

2.3 A `cleanup()` function?

Listing 4 `database_helper_with_cleanup.cpp`

```cpp
#include <iostream>
#include <string>

class DatabaseHelper {
  private:
    std::string* connection{};

  public:
    DatabaseHelper(const std::string& path)
        : connection{new std::string{path}} {
        std::cout << "Connected to " << *connection << '\n';
    }

    void cleanup() { // the user must remember to call this
        delete connection;
        connection = nullptr;
        std::cout << "Cleaned up\n";
    }
};

int main() {
    {
        DatabaseHelper db{"127.0.0.1:5432/my_database"};
        db.cleanup(); // forget this line, and memory leaks
    }
    return 0;
}
```

```
Connected to 127.0.0.1:5432/my_database
Cleaned up
```

It works, but **every** user must **remember** to call `cleanup()`. Someone, someday, will forget.

2.4 Destructors to the rescue

A **destructor** is a special member function that runs **automatically** when an object is destroyed, so the class can clean up after itself.

- Its name is the class name with a **tilde** in front: `~Simple()`.
- It takes **no arguments** and has **no return type**.
- A class has **exactly one** destructor.

Constructors set an object up; destructors tidy it away.

2.5 A first destructor

Listing 5 `destructor_example.cpp`

```cpp
#include <iostream>

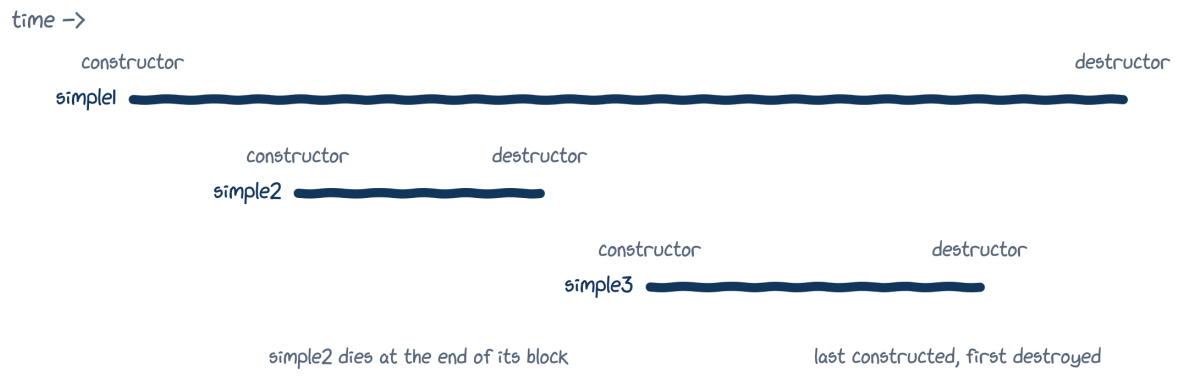
class Simple {
  private:
    int id{};

  public:
    Simple(int id) : id{id} { std::cout << "Constructing " << id << '\n'; }
    ~Simple() { std::cout << "Destructing " << id << '\n'; } // destructor
};

int main() {
    Simple simple1{1};
    {
        Simple simple2{2};
    } // simple2 is destroyed here
    Simple simple3{3};
    return 0;
} // simple3, then simple1, are destroyed here
```

```
Constructing 1
Constructing 2
Destructing 2
Constructing 3
Destructing 3
Destructing 1
```

2.6 When do destructors run?



- A local object is destroyed at the end of the block where it was created, just like local variables in Chapter 02.
- Objects in the same block are destroyed in **reverse** order of creation: last constructed, first destroyed.
- If you don't write a destructor, the compiler writes an empty **implicit destructor**. If there's nothing to clean up, that's all you need.

2.7 The database helper, fixed

Listing 6 `database_helper.cpp`

```cpp
#include <iostream>
#include <string>

class DatabaseHelper {
  private:
    std::string* connection{};

  public:
    DatabaseHelper(const std::string& path)
        : connection{new std::string{path}} {
        std::cout << "Connected to " << *connection << '\n';
    }

    ~DatabaseHelper() { // called automatically: nothing to remember
        std::cout << "Disconnecting from " << *connection << '\n';
        delete connection;
    }
};

int main() {
    {
        DatabaseHelper db{"127.0.0.1:5432/my_database"};
    } // the destructor runs right here
    std::cout << "db is gone, and so is its memory\n";
    return 0;
}
```

```
Connected to 127.0.0.1:5432/my_database
Disconnecting from 127.0.0.1:5432/my_database
db is gone, and so is its memory
```

Nothing to remember, and AddressSanitizer reports no leaks.

2.8 RAII: let objects clean up after themselves

The pattern we just used has a name: **RAII**, *Resource Acquisition Is Initialization* (a famously bad name for a great idea).

- **Acquire** a resource (memory, a file, a network connection) in the **constructor**.
- **Release** it in the **destructor**.

Then the resource lives exactly as long as the object, and cleanup can never be forgotten.

> **!** Key idea
>
> This is how `std::string`, `std::vector` and `std::unique_ptr` (Chapter 09) manage their memory. Now you know how it's done!

2.9 A class template that owns an array

Listing 7 `array_with_destructor.cpp`

```cpp
#include <iostream>

template <typename T, int N> // a type AND a value (Chapter 06)
class FixedArray {
    T* data{}; // private by default

  public:
    FixedArray(const T& value) : data{new T[N]} { // acquire...
        for (int i{0}; i < N; ++i) {              // i < N, not i <= N!
            data[i] = value;
        }
    }
    ~FixedArray() {    // ...and release
        delete[] data; // new[] needs delete[] (Chapter 09)
        std::cout << "Memory returned\n";
    }
    T get(int i) const { return data[i]; }
};

int main() {
    FixedArray<double, 5> numbers{4.2};
    std::cout << numbers.get(0) << ' ' << numbers.get(4) << '\n';
    return 0;
} // no cleanup() call needed
```

```
4.2 4.2
Memory returned
```

`FixedArray` is a **class template** with a type parameter `T` and a value parameter `N` (more on class templates shortly).

Note `delete[]`, and `i < N`: writing `data[N]` would be out of bounds.

2.10 Watch out: copying an object that owns memory

Listing 8 `shallow_copy_problem.cpp`

```cpp
#include <iostream>

class IntBox { // owns one int on the heap
  private:
    int* data{};

  public:
    IntBox(int value) : data{new int{value}} {}
    ~IntBox() { delete data; }

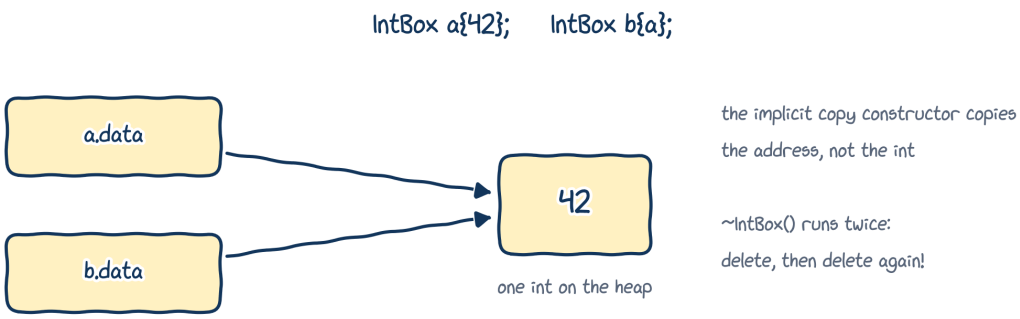
    int get() const { return *data; }
};

int main() {
    IntBox a{42};
    IntBox b{a}; // copies the POINTER, not the int
    std::cout << a.get() << ' ' << b.get() << '\n';
    return 0;
} // b's destructor deletes the int... then a's deletes it again!
```

```
$ ./shallow_copy_problem
42 42
free(): double free
detected in tcache 2
Aborted
```

It printed fine, then crashed on the way out. Why?

2.11 What went wrong: a shallow copy



The implicit copy constructor copies each member. The member is a **pointer**, so `b` gets the same address as `a`: a **shallow copy**. When `main()` ends, both destructors delete the same `int`. Undefined behaviour, and here a crash.

2.12 The rule of three (and of zero)

> Best practice: the rule of three
>
> If a class needs a **destructor**, it almost certainly also needs its own **copy constructor** and **copy assignment** (Chapter 13), or must forbid copying with `= delete` (Chapter 11).

Better still, follow the **rule of zero**: let members that already manage themselves (`std::vector`, `std::string`, `std::unique_ptr`) own the resources. Then you need no destructor, and the implicit copy does the right thing.

3 Class templates

3.1 The same problem, for classes

We met this problem for functions in Chapter 06:

```
struct PairInt {
    int first{};
    int second{};
};

struct PairDouble {
    double first{};
    double second{};
};
```

And a pair of `std::vector<double>`s? Of strings? You've already used the answer: `std::vector<double>` is a **class template**.

3.2 Class templates

A **class template** is a recipe for making class types, just as a function template is a recipe for functions.

Listing 9 `pair_class_template.cpp`

```cpp
#include <iostream>

template <typename T> // the template parameter declaration
struct Pair {
    T first{};
    T second{};
};

int main() {
    Pair<int> p1{5, 6}; // instantiates Pair<int>
    std::cout << p1.first << ' ' << p1.second << '\n';

    Pair<double> p2{1.2, 3.4}; // instantiates Pair<double>
    std::cout << p2.first << ' ' << p2.second << '\n';

    Pair<double> p3{7.8, 9.0}; // reuses Pair<double>
    std::cout << p3.first << ' ' << p3.second << '\n';
    return 0;
}
```

```
5 6
1.2 3.4
7.8 9
```

Start with `template <typename T>`, then write the `struct` or `class` as usual, using `T`. Use it as `Pair<int>`, `Pair<double>`, …

3.3 What the compiler generates

Listing 10 `pair_class_template_compiler_pov.cpp`

```cpp
#include <iostream>

template <typename T>
struct Pair; // only the declaration is left

template <> // what the compiler generates for T = int...
struct Pair<int> {
    int first{};
    int second{};
};

template <> // ...and for T = double
struct Pair<double> {
    double first{};
    double second{};
};

int main() {
    Pair<int> p1{5, 6};
    std::cout << p1.first << ' ' << p1.second << '\n';
    Pair<double> p2{1.2, 3.4};
    std::cout << p2.first << ' ' << p2.second << '\n';
    return 0;
}
```

```
5 6
1.2 3.4
```

As with function templates, the compiler stamps out one real class per type you use, `Pair<int>` and `Pair<double>`. You never write `template <>` like this yourself: it just shows you what happens.

3.4 Class templates and function templates together

Listing 11 `class_template_and_func_template.cpp`

```cpp
#include <iostream>

template <typename T>
struct Pair {
    T first{};
    T second{};
};

template <typename T>
constexpr T max(Pair<T> p) { // a function template taking a Pair<T>
    return (p.first < p.second) ? p.second : p.first;
}

int main() {
    Pair<int> p1{5, 6};
    std::cout << max<int>(p1) << " is larger\n"; // T given explicitly

    Pair<double> p2{1.2, 3.4};
    std::cout << max(p2) << " is larger\n"; // T deduced (prefer this)
    return 0;
}
```

```
6 is larger
3.4 is larger
```

For the call `max(p1)`, the compiler instantiates `Pair<int>` *and* the function `int max<int>(Pair<int> p)`. `constexpr` (Chapter 06) lets the compiler run `max()` at compile time when the arguments are known then.

3.5 Several template types

Listing 12 `class_multiple_template_types.cpp`

```cpp
#include <iostream>

template <typename T, typename U>
struct Pair {
    T first{};
    U second{};
};

template <typename T, typename U>
void print(const Pair<T, U>& p) {
    std::cout << '[' << p.first << ", " << p.second << "]\n";
}

int main() {
    Pair<int, double> p1{1, 2.3}; // an int and a double
    Pair<double, int> p2{4.5, 6}; // a double and an int
    Pair<int, int> p3{7, 8};      // two ints
    print(p1);
    print(p2);
    print(p3);
    return 0;
}
```

```
[1, 2.3]
[4.5, 6]
[7, 8]
```

3.6 `std::pair`: the standard library's version

Pairs of values are so common that the standard library has exactly this class template, `std::pair` in `<utility>`, with members `first` and `second`:

Listing 13 `std_pair_example.cpp`

```cpp
#include <iostream>
#include <utility> // for std::pair

auto max(const auto& pair) { // abbreviated function template (Ch. 06)
    return (pair.first < pair.second) ? pair.second : pair.first;
}

int main() {
    std::pair<int, double> p1{1, 2.3};
    std::pair<double, int> p2{4.5, 6};
    std::cout << "max(p1) = " << max(p1) << '\n';
    std::cout << "max(p2) = " << max(p2) << '\n';
    std::cout << p1.first << ' ' << p1.second << '\n';
    return 0;
}
```

```
max(p1) = 2.3
max(p2) = 6
1 2.3
```

4 Friends

4.1 Why friends?

You've written `std::cout << x;` countless times. `<<` is an operator with two operands, `std::cout` and `x`, much like `a + b`.

For our own classes, we'd love to write `std::cout << polynomial;`. The function behind that `<<` needs to see the polynomial's **private** data, but it can't be a member of our class (Chapter 13 explains why).

C++'s answer: make it a **friend**.

4.2 Friendship

- Inside a class, a **friend declaration** (`friend ...;`) names a function or class that gets **full access** to this class's private (and protected) members.
- That function or class is a **friend**.

> **!** Key idea
>
> Friendship is always **granted** by the class whose members will be accessed. It can't be claimed by the function or class that wants access.

4.3 A friend non-member function

Listing 14 `friend_nonmember_function.cpp`

```cpp
#include <iostream>

class Accumulator {
  private:
    double value{};

  public:
    void add(double x) { value += x; }

    // print() is not a member, but it may use our private members
    friend void print(const Accumulator& accumulator);
};

void print(const Accumulator& accumulator) { // not a member!
    std::cout << accumulator.value << '\n';  // allowed: it's a friend
}

int main() {
    Accumulator acc{};
    acc.add(42);
    print(acc); // called like any non-member function
    return 0;
}
```

```
42
```

`print()` is an ordinary non-member function, but `Accumulator` declared it a friend, so it may read `accumulator.value`.

4.4 Defining a friend inside the class

Listing 15 `friend_nonmember_function_inside_class.cpp`

```cpp
#include <iostream>

class Accumulator {
  private:
    double value{};

  public:
    void add(double x) { value += x; }

    // Defined inside the class, but still NOT a member function
    friend void print(const Accumulator& accumulator) {
        std::cout << accumulator.value << '\n';
    }
};

int main() {
    Accumulator acc{};
    acc.add(5);
    print(acc); // not acc.print()
    return 0;
}
```

```
5
```

Even though `print()` is defined inside `Accumulator`, it is **not** a member: we call it as `print(acc)`, not `acc.print()`.

4.5 A friend of several classes

Listing 16 `function_friend_with_multiple_classes.cpp`

```cpp
#include <iostream>

class Humidity;     // forward declaration: used before it's defined
class Temperature { // members are private by default
    int temp{};

  public:
    Temperature(int temp) : temp{temp} {}
    friend void print_weather(const Temperature& t, const Humidity& h);
};

class Humidity {
    int humidity{};

  public:
    Humidity(int humidity) : humidity{humidity} {}
    friend void print_weather(const Temperature& t, const Humidity& h);
};

void print_weather(const Temperature& t, const Humidity& h) {
    std::cout << t.temp << " degrees, " << h.humidity << "% humidity\n";
}

int main() {
    print_weather(Temperature{12}, Humidity{10});
    return 0;
}
```

```
12 degrees, 10% humidity
```

`class Humidity;` is a forward declaration of a class, needed because `Temperature` mentions `Humidity` before it's defined.

4.6 Prefer non-friend functions

Listing 17 `prefer_nonfriend_to_friend_01.cpp`

```cpp
#include <iostream>

class Accumulator {
  private:
    int value{};

  public:
    void add(int x) { value += x; }
    friend void print(const Accumulator& acc);
};

void print(const Accumulator& acc) { std::cout << acc.value << '\n'; }

int main() {
    Accumulator acc{};
    acc.add(5);
    print(acc);
    return 0;
}
```

```
5
```

This works, but `print()` depends on a private detail. Rename `value`, and `print()` breaks too.

4.7 ...when a public member will do

Listing 18 `prefer_nonfriend_to_friend_02.cpp`

```cpp
#include <iostream>

class Accumulator {
  private:
    int value{};

  public:
    void add(int x) { value += x; }
    int get_value() const { return value; }
};

void print(const Accumulator& acc) { std::cout << acc.get_value() << '\n'; }

int main() {
    Accumulator acc{};
    acc.add(5);
    print(acc);
    return 0;
}
```

```
5
```

With a public getter, `print()` needs no friendship, and doesn't care what the member is called.

> **Best practice**
>
> Prefer non-friend functions when the public interface is enough.

4.8 Friend classes

Listing 19 `friend_class_example.cpp`

```cpp
#include <iostream>

class Storage {
  private:
    int int_value{};
    double double_value{};

  public:
    Storage(int i, double d) : int_value{i}, double_value{d} {}
    friend class Display; // Display may use our private members
};

class Display {
  public:
    void show(const Storage& s) const { // allowed: Display is a friend
        std::cout << s.int_value << ' ' << s.double_value << '\n';
    }
};

int main() {
    Storage storage{5, 6.7};
    Display display{};
    display.show(storage);
    return 0;
}
```

```
5 6.7
```

A **friend class** may use the private members of the class that befriends it: every member function of `Display` can read a `Storage`'s private data.

4.9 Friendship has limits

1. **It isn't mutual.** `Display` is a friend of `Storage`, but `Storage` is not a friend of `Display`, unless `Display` says so too.
2. **It isn't transitive.** If `A` is a friend of `B`, and `B` is a friend of `C`, `A` is still not a friend of `C`.
3. **It isn't inherited.** If `A` befriends `B`, classes derived from `B` (Chapter 14) are not friends of `A`.

4.10 Friend member functions (self-study)

Instead of befriending a whole class, you can befriend a single **member function**. It takes some care with the order of definitions, so this part is for **self-study**. We'll build it up in three attempts.

4.11 Attempt 1: the friend's class isn't defined yet

Listing 20 `friend_member_function_01.cpp`

```cpp
#include <iostream>

class Display; // only a forward declaration

class Storage {
  private:
    int int_value{};
    double double_value{};

  public:
    Storage(int i, double d) : int_value{i}, double_value{d} {}
    friend void Display::show(const Storage& s) const; // error!
};

class Display {
  public:
    void show(const Storage& s) const {
        std::cout << s.int_value << ' ' << s.double_value << '\n';
    }
};

int main() {
    Display{}.show(Storage{5, 6.7});
    return 0;
}
```

```
error: invalid use of incomplete type 'class Display'
```

To befriend `Display::show`, the compiler must have seen the **full definition** of `Display`, not just a forward declaration.

4.12 Attempt 2: swap the classes around

Listing 21 `friend_member_function_02.cpp`

```cpp
#include <iostream>

class Display {
  public:
    void show(const Storage& s) const { // error: what is a Storage?
        std::cout << s.int_value << ' ' << s.double_value << '\n';
    }
};

class Storage {
  private:
    int int_value{};
    double double_value{};

  public:
    Storage(int i, double d) : int_value{i}, double_value{d} {}
    friend void Display::show(const Storage& s) const;
};

int main() {
    Display{}.show(Storage{5, 6.7});
    return 0;
}
```

```
error: 'Storage' does not name a type
```

Now `Display::show()` mentions `Storage` before the compiler has heard of it.

4.13 Attempt 3: declare first, define later

Listing 22 `friend_member_function_03.cpp`

```cpp
#include <iostream>

class Storage; // 1. a forward declaration of Storage

class Display {
  public:
    void show(const Storage& s) const; // 2. declared, not defined yet
};

class Storage {
  private:
    int int_value{};
    double double_value{};

  public:
    Storage(int i, double d) : int_value{i}, double_value{d} {}
    friend void Display::show(const Storage& s) const; // fine now
};

void Display::show(const Storage& s) const { // 3. defined after Storage
    std::cout << s.int_value << ' ' << s.double_value << '\n';
}

int main() {
    Display{}.show(Storage{5, 6.7});
    return 0;
}
```

```
5 6.7
```

1. Forward-declare `Storage`.
2. *Declare* `show()`.
3. *Define* it after `Storage`.

5 Wrapping up

5.1 Summary

- **`this`** points at the implicit object. Write it when a parameter hides a member, and return `*this` by reference to allow **method chaining**.
- A **destructor** `~ClassName()` runs automatically when an object is destroyed. Objects in a block are destroyed in reverse order.
- **RAII**: acquire resources in the constructor and release them in the destructor. If you write a destructor, think about copying (**rule of three**); better, own nothing by hand (**rule of zero**).
- A **class template** is a recipe for classes: `Pair<int>`, `std::pair<int, double>`, `std::vector<double>`.
- A **friend** function or class may use a class's private members. Friendship is granted, not taken. Prefer non-friends when public members are enough.

5.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. The hidden `this` pointer and member function chaining
2. Destructors, and the rule of three
3. Class templates
4. Friend classes and friend member functions

Next up, Chapter 13: operator overloading. We'll finally write our own `+` and `<<`.